

Chương 2 đã phân tích yêu cầu và mô tả các chức năng của hệ thống. Chương này trình bày các công nghệ và nền tảng được sử dụng để hiện thực các yêu cầu đó, phân tích vai trò của từng thành phần trong kiến trúc tổng thể, và giải thích lý do lựa chọn so với các phương án thay thế. Các công nghệ chính gồm SUMO, TraCI, Python và Unity; ngoài ra chương cũng trình bày cách tích hợp các thành phần này thành một hệ thống mô phỏng–hiển thị thống nhất.

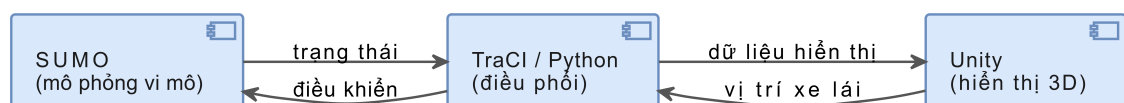
0.1 Tổng quan công nghệ và vai trò trong hệ thống

Bài toán cần giải quyết là hiển thị giao thông trong không gian ba chiều dựa trên dữ liệu mô phỏng, đồng thời cho phép tương tác. Để đạt được mục tiêu này, hệ thống cần đáp ứng đồng thời bốn nhóm yêu cầu: (i) mô phỏng giao thông vi mô theo thời gian, (ii) trích xuất và điều khiển mô phỏng theo từng bước thời gian, (iii) trực quan hóa kết quả dưới dạng ba chiều có khả năng quan sát và tương tác, và (iv) xử lý vật lý cho phương tiện khi người dùng lái thử công.

Ngăn xếp công nghệ được lựa chọn gồm bốn thành phần chính:

- **SUMO** (Simulation of Urban Mobility): nền tảng mô phỏng giao thông đô thị mã nguồn mở, hỗ trợ mô phỏng vi mô theo từng phương tiện.
- **TraCI** (Traffic Control Interface): giao diện cho phép chương trình bên ngoài kết nối, truy vấn và điều khiển SUMO theo từng bước thời gian.
- **Python**: lớp điều phối chịu trách nhiệm khởi tạo mô phỏng, giao tiếp qua TraCI, chuẩn hóa và truyền dữ liệu.
- **Unity**: môi trường dựng cảnh, hiển thị ba chiều theo thời gian thực và xử lý tương tác, vật lý.

Về mặt kiến trúc, luồng dữ liệu tổng quát có thể mô tả như sau: SUMO mô phỏng giao thông và cung cấp trạng thái đối tượng theo thời gian; Python dùng TraCI lấy trạng thái này theo từng bước và truyền sang Unity theo một định dạng chuẩn; Unity đọc dữ liệu, ánh xạ hệ tọa độ và dựng cảnh. Khi người dùng chiếm quyền điều khiển một phương tiện, dữ liệu còn đi theo chiều ngược lại: Unity gửi vị trí phương tiện do người dùng lái về Python để phản ánh vào SUMO. Hình 0.1 minh họa kiến trúc tổng quát này.



Hình 0.1: Kiến trúc tổng quát hệ thống mô phỏng và hiển thị 3D

0.2 SUMO (Simulation of Urban Mobility)

0.2.1 Giới thiệu

SUMO là công cụ mô phỏng giao thông đô thị mã nguồn mở, được thiết kế để mô phỏng giao thông theo mô hình vi mô **sumo2012**, **sumo_web**. Trong mô hình vi mô, mỗi phương tiện được xem như một thực thể có trạng thái riêng gồm vị trí, vận tốc, gia tốc và hướng di chuyển, tuân theo các quy tắc chuyển động, tuân thủ làn đường và điều khiển giao lộ. Cách tiếp cận này phù hợp khi mục tiêu là tái hiện chuyển động phương tiện liên tục theo thời gian và phục vụ trực quan hóa.

Trong phạm vi đồ án, SUMO được sử dụng để tạo ra dữ liệu mô phỏng nhất quán theo thời gian. Thay vì chỉ xuất báo cáo thống kê tổng hợp, hệ thống cần lấy trạng thái chi tiết của từng phương tiện theo từng bước thời gian để hiển thị chuyển động trong Unity.

0.2.2 Lý do lựa chọn SUMO

SUMO được lựa chọn vì các lý do sau:

- **Phù hợp bài toán mô phỏng đô thị:** hỗ trợ mạng đường dạng đô thị nút–cạnh, mô phỏng làn đường, giao lộ và tín hiệu đèn.
- **Khả năng điều khiển theo thời gian:** kết hợp với TraCI cho phép chạy mô phỏng theo từng bước và truy vấn trạng thái tức thời.
- **Dễ tích hợp:** định dạng tệp rõ ràng, hệ sinh thái công cụ phong phú, thuận tiện để Python xử lý và tự động hóa.
- **Mã nguồn mở:** phù hợp với mục tiêu học thuật và triển khai trong khuôn khổ đồ án.

0.2.3 Các thành phần cấu hình và dữ liệu trong SUMO

Để chạy một kịch bản, SUMO cần ba nhóm tệp chính: tệp mô tả mạng đường, tệp mô tả luồng/tuyến phương tiện và tệp cấu hình mô phỏng.

Mạng đường. Mạng đường thể hiện hạ tầng giao thông ở dạng đồ thị có hướng gồm nút và cạnh. Mạng có thể bắt đầu từ các tệp rời mô tả nút và cạnh, sau đó biên dịch thành tệp mạng hoàn chỉnh chứa thông tin làn đường, tọa độ và kết nối tại nút giao.

Luồng/tuyến. Luồng phương tiện được mô tả trong tệp tuyến đường, trong đó định nghĩa tuyến, loại phương tiện và thời điểm khởi hành. Khi cần tái hiện giao thông có mật độ thay đổi, cách xây dựng tuyến cần điều chỉnh được lưu lượng và phân bố.

Cấu hình chạy. Tệp cấu hình tập hợp các thành phần trên và chỉ định tham số

mô phỏng như thời gian bắt đầu, kết thúc, độ dài bước thời gian và chế độ xuất dữ liệu. Việc chuẩn hóa tham số giúp các thí nghiệm có thể chạy lại và so sánh.

Bảng 1: Đầu vào và đầu ra của SUMO trong hệ thống

Nhóm	Mô tả
Đầu vào	Mạng đường, luồng/tuyến, tệp cấu hình mô phỏng
Đầu ra	Trạng thái theo thời gian: danh sách phương tiện, vị trí, vận tốc, hướng; trạng thái đèn; thông tin làn/cạnh khi cần

0.2.4 Các lựa chọn thay thế và đánh giá

Với bài toán mô phỏng giao thông, có thể xem xét nhiều công cụ khác. Một số công cụ thương mại như Aimsun hoặc VISSIM có khả năng mô phỏng mạnh mẽ nhưng đi kèm hạn chế về giấy phép và chi phí trong khuôn khổ đề án. Ở chiều ngược lại, các công cụ mô phỏng mã nguồn mở tập trung vào lưu lượng tổng thể, không tối ưu cho việc lấy trạng thái chi tiết của từng phương tiện để hiển thị ba chiều liên tục. Các nền tảng ba chiều phục vụ xe tự hành như CARLA cung cấp mô phỏng cảm biến nhưng yêu cầu tài nguyên lớn hơn nhiều so với mục tiêu hiển thị dòng giao thông dựa trên dữ liệu mô phỏng.

0.3 TraCI (Traffic Control Interface)

0.3.1 Giới thiệu

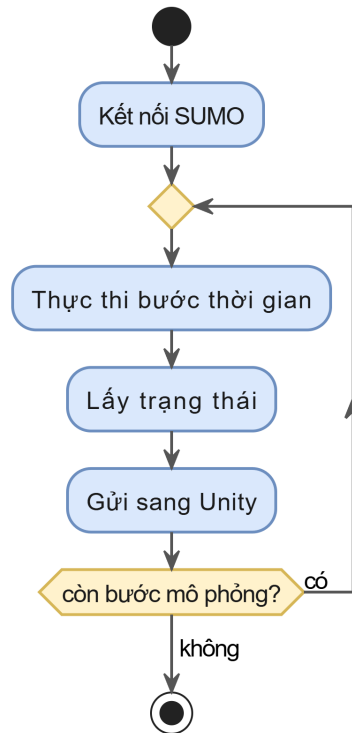
TraCI là giao diện cho phép một chương trình bên ngoài kết nối tới SUMO để điều khiển mô phỏng theo từng bước thời gian **traci2008**. Thay vì để SUMO tự chạy độc lập và chỉ xuất nhật ký sau khi kết thúc, TraCI cho phép hệ thống điều khiển tiến trình mô phỏng theo bước, truy vấn trạng thái đối tượng tại thời điểm đang chạy, thay đổi trạng thái hoặc ra lệnh điều khiển, và đồng bộ dữ liệu với hệ thống hiển thị.

Trong đề án, TraCI là cầu nối quan trọng để Python lấy dữ liệu theo thời gian và cung cấp cho Unity. Nhờ đó, Unity không cần đọc các tệp nhật ký hậu kỳ mà có thể cập nhật chuyển động theo nhịp mô phỏng.

0.3.2 Mô hình hoạt động máy chủ–máy khách

Cần lưu ý hệ thống có hai quan hệ máy chủ–máy khách tách biệt, không nên nhầm lẫn. Trong quan hệ với SUMO thông qua TraCI, SUMO là máy chủ còn Python là máy khách. Ngược lại, trong quan hệ truyền dữ liệu với Unity (trình bày ở Mục 0.6.1), Python lại đóng vai trò máy chủ còn Unity là máy khách. Như vậy Python giữ vai trò kép: là máy khách của SUMO để điều khiển mô phỏng, đồng thời là máy chủ của Unity để cung cấp dữ liệu hiển thị.

Xét riêng quan hệ với SUMO: SUMO chạy như một máy chủ, còn Python với vai trò máy khách khởi tạo kết nối, gửi lệnh thực thi bước thời gian, sau đó truy vấn các đối tượng quan tâm như danh sách xe đang tồn tại, vị trí từng xe và trạng thái đèn tín hiệu. Mỗi vòng lặp tương ứng với một bước thời gian, từ đó hình thành luồng dữ liệu theo chuỗi thời gian. Hình 0.2 minh họa vòng lặp này.



Hình 0.2: Vòng lặp điều khiển và thu thập dữ liệu bằng TraCI

0.3.3 Nhóm chức năng TraCI sử dụng trong đồ án

Tùy theo kịch bản và dữ liệu cần hiển thị, TraCI cung cấp nhiều nhóm hàm truy vấn và điều khiển. Trong đồ án, các nhóm thường dùng gồm:

- **Mô phỏng:** điều khiển tiến trình, lấy thời gian mô phỏng và số xe đang tồn tại.
- **Phương tiện:** lấy và cập nhật trạng thái xe gồm tọa độ, tốc độ, hướng; truy vấn tuyến và làn hiện tại. Nhóm này cũng được dùng khi người dùng chiếm quyền điều khiển để phản ánh vị trí xe do người lái vào mô phỏng, và để xóa xe khỏi mô phỏng khi người dùng trả quyền.
- **Đèn tín hiệu:** lấy trạng thái pha đèn của từng nút giao.
- **Cạnh/làn:** lấy thông tin hạ tầng và hình học làn đường phục vụ hiển thị.

0.3.4 Lựa chọn thay thế

Một cách tiếp cận thay thế là chạy SUMO và xuất nhật ký sau mô phỏng, rồi để Unity đọc lại và phát lại. Cách này đơn giản, dễ kiểm tra, nhưng hạn chế khả năng đồng bộ theo thời gian và khó mở rộng sang tình huống cần can thiệp mô phỏng trong lúc chạy, ví dụ khi người dùng lái xe và tác động lên dòng giao thông. Do đó, TraCI phù hợp hơn với mục tiêu kết nối mô phỏng và hiển thị có tương tác. Trên thực tế, đồ án dùng TraCI cho chế độ thời gian thực và dùng cách phát lại từ tệp cho chế độ phát lại.

0.4 Python cho điều phối và xử lý dữ liệu

0.4.1 Vai trò trong hệ thống

Python đóng vai trò lớp điều phối giữa mô phỏng và hiển thị. Ở mức hệ thống, Python chịu trách nhiệm khởi tạo phiên làm việc với SUMO, điều khiển mô phỏng theo từng bước qua TraCI, thu thập dữ liệu cần thiết, chuẩn hóa dữ liệu theo một định dạng trao đổi thống nhất, và truyền dữ liệu sang Unity. So với việc viết toàn bộ lớp điều phối trong Unity, Python mang lại lợi thế về tốc độ phát triển, khả năng thử nghiệm nhanh và thuận tiện cho việc viết kịch bản tự động tạo dữ liệu.

0.4.2 Chuẩn hóa theo định dạng trao đổi độc lập nền tảng

Một mục tiêu quan trọng là tách biệt tầng mô phỏng và thu thập dữ liệu khỏi tầng hiển thị. Vì vậy, Python xuất dữ liệu theo một định dạng trao đổi có thể tái sử dụng bởi nhiều nền tảng đồ họa khác nhau, thống nhất các nội dung tối thiểu như đơn vị đo, hệ tọa độ nguồn, các trường dữ liệu bắt buộc và cấu trúc bản ghi. Nhờ cách thiết kế này, khi thay đổi nền tảng hiển thị, phần Python có thể giữ nguyên, chỉ cần xây dựng một tầng đọc dữ liệu mới ở phía hiển thị.

0.4.3 Tổ chức xử lý theo bước thời gian

Python chạy vòng lặp theo từng bước: mỗi bước tiến mô phỏng một khoảng thời gian, sau đó lấy trạng thái và gửi sang Unity. Trong các bước này có thể áp dụng tối ưu như chỉ gửi các phương tiện đang tồn tại. Để chuyển động mượt trong Unity, hệ thống lựa chọn nội suy ở phía hiển thị thay vì tăng tần suất gửi dữ liệu.

0.4.4 Lựa chọn thay thế

Về mặt kỹ thuật, TraCI cũng hỗ trợ máy khách viết bằng Java hoặc C++. Tuy nhiên, với đồ án ứng dụng, Python là lựa chọn cân bằng giữa tính linh hoạt, khả năng xử lý dữ liệu và thời gian phát triển. Viết trực tiếp lớp giao tiếp trong Unity sẽ giảm một tầng trung gian nhưng làm tăng độ phức tạp và khó tái sử dụng các kịch bản xử lý dữ liệu.

0.5 Unity cho hiển thị 3D và tương tác

0.5.1 Giới thiệu và lý do lựa chọn

Unity là nền tảng phát triển ứng dụng ba chiều thời gian thực, hỗ trợ dựng cảnh, điều khiển camera, chiếu sáng, vật lý và xây dựng giao diện tương tác **unity_manual**. Trong đồ án, Unity đảm nhiệm trực quan hóa giao thông ba chiều dựa trên dữ liệu mô phỏng, cho phép người dùng quan sát mật độ xe, dòng xe qua nút giao và hành vi di chuyển theo thời gian. Unity được lựa chọn vì có hệ sinh thái phong phú cho dựng cảnh và xử lý chuyển động, khả năng cập nhật đối tượng theo thời gian thực, hệ thống vật lý sẵn có phục vụ chế độ lái, và phù hợp với yêu cầu triển khai nhanh.

0.5.2 Pipeline hiển thị đối tượng giao thông

Để hiển thị một mô phỏng giao thông, Unity quản lý hai nhóm đối tượng: hạ tầng gồm đường, làn, nút giao; và đối tượng động gồm phương tiện và người đi bộ. Một pipeline hiển thị thường gồm các bước: khởi tạo hạ tầng dựa trên dữ liệu mạng; khởi tạo phương tiện theo danh sách và gán mô hình; và cập nhật vị trí, hướng theo dữ liệu nhận được tại mỗi bước. Khi số lượng xe lớn, hệ thống áp dụng tái sử dụng đối tượng và chỉ hiển thị trong phạm vi quan sát để duy trì hiệu năng.

0.5.3 Hệ thống vật lý của Unity cho chế độ lái

Khi người dùng chiếm quyền điều khiển một phương tiện, phương tiện đó chuyển từ chế độ được cập nhật vị trí theo dữ liệu mô phỏng sang chế độ chuyển động theo vật lý. Unity cung cấp các thành phần cần thiết cho chế độ này. Thành phần thân cứng (Rigidbody) **unity_rigidbody** cho phép phương tiện chịu tác động của lực, va chạm và quán tính. Thành phần bộ va chạm bánh xe (Wheel Collider) **unity_wheelcollider** mô phỏng lực kéo, lực phanh và độ bám của từng bánh, qua đó tái hiện chuyển động lái có tính vật lý thay vì dịch chuyển tức thời. Khi phương tiện không do người dùng điều khiển, hệ thống tắt phần vật lý và chuyển về chế độ cập nhật vị trí theo dữ liệu để bảo đảm hiệu năng. Cơ chế chuyển đổi giữa hai chế độ này là một điểm kỹ thuật quan trọng, được trình bày chi tiết trong Chương 5.

0.5.4 Lựa chọn thay thế

Các nền tảng ba chiều khác như Unreal Engine có khả năng đồ họa mạnh nhưng thường yêu cầu tài nguyên lớn và chu trình phát triển nặng hơn. Các giải pháp trên nền web thuận lợi cho triển khai trực tuyến nhưng cần tự xây dựng nhiều thành phần engine và tối ưu hiệu năng. Vì vậy Unity là lựa chọn phù hợp cho mục tiêu trực quan hóa ba chiều có tương tác trong đồ án.

0.6 Tích hợp SUMO–TraCI/Python–Unity

0.6.1 Phương thức trao đổi dữ liệu

Trong đồ án, dữ liệu được truyền theo thời gian thực bằng giao thức TCP **rfc793**. Ở quan hệ này, Python đóng vai trò máy chủ và phía gửi, còn Unity đóng vai trò máy khách và phía nhận, liên tục đọc gói dữ liệu và cập nhật trạng thái hiển thị. Đây chính là quan hệ máy chủ–máy khách thứ hai đã nêu ở Mục 0.3.2, và ngược chiều với quan hệ giữa Python và SUMO. Cách truyền trực tiếp qua TCP có độ trễ thấp, đồng bộ tốt với tiến trình mô phỏng theo từng bước. Khi người dùng lái xe, một kênh truyền theo chiều ngược lại được dùng để Unity gửi vị trí phương tiện do người lái về phía Python.

Vì TCP là giao thức hướng luồng, một lần nhận có thể chứa một phần hoặc nhiều bản tin gộp lại. Do đó, hệ thống bổ sung cơ chế tách khung bản tin bằng một dấu hiệu kết thúc đặt ở cuối mỗi bản tin; chi tiết được trình bày trong Chương 5.

0.6.2 Định dạng dữ liệu tối thiểu

Để hiển thị chuyển động phương tiện, Unity tối thiểu cần biết định danh xe, vị trí, hướng và vận tốc tại mỗi mốc thời gian. Ngoài ra, để phục vụ phân tích và mở rộng, dữ liệu có thể kèm theo thông tin làn/cạnh hoặc trạng thái đèn. Định dạng này được thiết kế độc lập với engine: các trường phản ánh trạng thái mô phỏng và hình học nguồn, còn các bước chuyển đổi sang hệ trục ba chiều, tỉ lệ hiển thị và nội suy theo khung hình do phía hiển thị đảm nhiệm.

Bảng 2: Một số trường dữ liệu trao đổi giữa Python và Unity

Trường dữ liệu	Ý nghĩa
Định danh	Định danh duy nhất của phương tiện trong mô phỏng
Vị trí	Tọa độ của phương tiện trong hệ của SUMO
Hướng	Hướng di chuyển để xoay mô hình ba chiều
Vận tốc	Tốc độ của phương tiện
Trạng thái đèn	Trạng thái đỏ/vàng/xanh của đèn tín hiệu

0.6.3 Đồng bộ thời gian giữa bước mô phỏng và khung hình

SUMO vận hành theo bước thời gian rời rạc với độ dài cố định, trong khi Unity cập nhật theo số khung hình mỗi giây và có thể thay đổi theo cấu hình máy. Nếu Unity chỉ cập nhật vị trí xe đúng tại mỗi bước, chuyển động có thể bị giật khi tốc độ khung hình cao. Có hai cách tiếp cận: cập nhật đúng theo bước, đơn giản và chính xác; hoặc nội suy theo khung hình, trong đó Unity nhận hai trạng thái liên tiếp và nội suy vị trí cho các khung hình ở giữa để chuyển động mượt hơn. Đồ án lựa chọn nội suy theo khung hình ở phía hiển thị nhằm bảo đảm chuyển động mượt và ổn

định.

0.6.4 Kết chương

Chương này đã trình bày các công nghệ chính được sử dụng trong đồ án gồm SUMO, TraCI, Python và Unity, cùng vai trò của từng thành phần trong kiến trúc tổng thể. Chương cũng phân tích cách tích hợp giữa mô phỏng và hiển thị ba chiều, đặc biệt là vấn đề trao đổi dữ liệu hai chiều và đồng bộ thời gian, và giới thiệu hệ thống vật lý của Unity phục vụ chế độ lái. Đây là nền tảng cho phần phân tích thiết kế và triển khai hệ thống ở Chương 4.